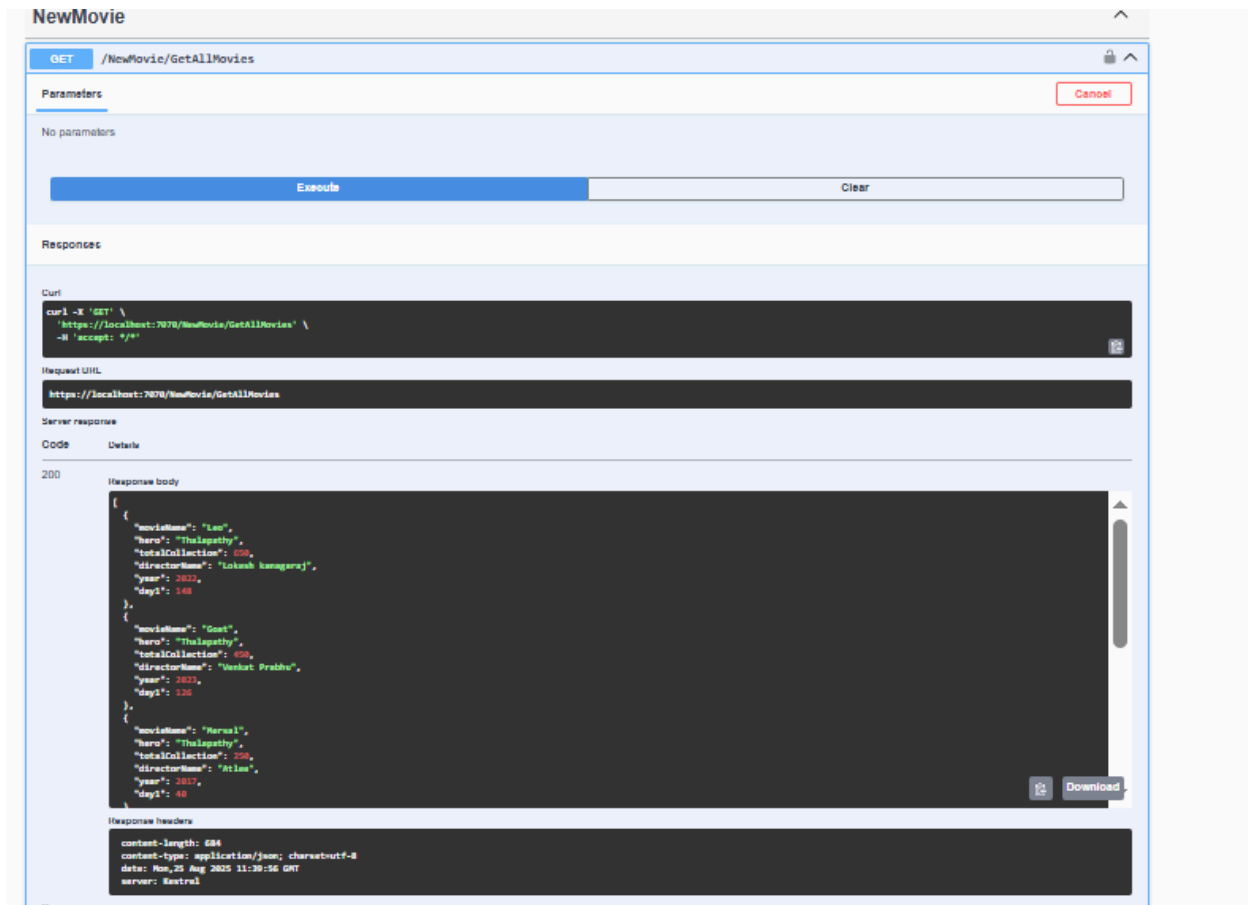


OutPut:



Controller:

using AutoMapper;  
using Microsoft.AspNetCore.Mvc;  
using SampleApi.DTO;  
using SampleApi.Interface;

namespace SampleApi.Controllers

```
{
    [Route("NewMovie")]
    [ApiController]
    public class NewMovieController : ControllerBase
    {
        private readonly INewMovie _newMovie;

        public NewMovieController(INewMovie newMovie)
        {
            _newMovie = newMovie;
        }
    }
}
```

```
[HttpGet("GetAllMovies")]
public IActionResult GetAllMovies()
{
    var movies = _newMovie.GetAllMovies();
    if (movies == null || movies.Count == 0)
        return NotFound("No movies found.");

    return Ok(movies);
}

[HttpGet("GetMoviesById/{id}")]
public IActionResult GetMoviesById(int id)
{
    var movie = _newMovie.GetMoviesById(id);
    if (movie == null)
        return NotFound($"Movie with ID {id} not found.");

    return Ok(movie);
}

[HttpPost("AddNewMovie")]
public IActionResult AddNewMovie([FromBody] NewMovieDTO movie)
{
    if (movie == null)
        return BadRequest("Invalid movie data.");

    var isAdded = _newMovie.AddNewMovie(movie);
    if (!isAdded)
        return StatusCode(500, "Failed to add movie.");

    return Ok("Movie added successfully.");
}

[HttpDelete("DeleteMovie/{id}")]
public IActionResult DeleteById(int id)
{
    var isDeleted = _newMovie.DeleteById(id);
    if (!isDeleted)
        return NotFound($"Movie with ID {id} not found.");

    return Ok("Movie deleted successfully.");
}
}
```

Interface:

```
using Microsoft.AspNetCore.Mvc;  
using SampleApi.DTO;
```

```
namespace SampleApi.Interface  
{  
    public interface INewMovie  
    {  
        public List<NewMovieDTO> GetAllMovies();  
        public NewMovieDTO GetMoviesByID(int id);  
        public bool AddNewMovie(NewMovieDTO movie);  
  
        public bool DeleteByID(int Id);  
    }  
}
```

Service:

```
using AutoMapper;  
using Microsoft.EntityFrameworkCore;  
using SampleApi.Context;  
using SampleApi.DTO;  
using SampleApi.Interface;  
using SampleApi.Model;
```

```
namespace SampleApi.Service  
{  
    public class NewMovieService : INewMovie  
    {  
        private readonly MyContext _context;  
        private readonly IMapper _mapper;  
  
        public NewMovieService(MyContext myContext, IMapper mapper)  
        {  
            _context = myContext;  
            _mapper = mapper;  
        }  
  
        public bool AddNewMovie(NewMovieDTO movie)  
        {  
            try  
            {
```

```
        var director = _context.directors.FirstOrDefault(d => d.DirectorName ==
movie.DirectorName);
        if (director == null)
        {
            director = new Director { DirectorName = movie.DirectorName };
            _context.directors.Add(director);
            _context.SaveChanges();
        }

        var collection = new Collection
        {
            Day1 = movie.Day1,
            year = movie.Year
        };
        _context.collections.Add(collection);
        _context.SaveChanges();

        var movietoadd = new NewMovie
        {
            Title = movie.MovieName,
            Hero = movie.Hero,
            TotalCollection = movie.TotalCollection,
            DirectorId = director.DirectorId,
            CollectionId = collection.CollectionId
        };

        _context.NewMovies.Add(movietoadd);
        _context.SaveChanges();

        return true;
    }
    catch
    {
        return false;
    }
}

public bool DeleteByID(int id)
{
    try
    {
        var movieToDelete = _context.NewMovies.FirstOrDefault(m => m.Id == id);
        if (movieToDelete == null) return false;
    }
}
```

```
        _context.NewMovies.Remove(movieToDelete);
        _context.SaveChanges();
        return true;
    }
    catch
    {
        return false;
    }
}

public List<NewMovieDTO> GetAllMovies()
{
    var movies = _context.NewMovies
        .Include(m => m.director)
        .Include(m => m.collection)
        .ToList();

    return _mapper.Map<List<NewMovieDTO>>>(movies);
}

public NewMovieDTO? GetMoviesByID(int id)
{
    var movie = _context.NewMovies
        .Include(m => m.director)
        .Include(m => m.collection)
        .FirstOrDefault(m => m.Id == id);

    return _mapper.Map<NewMovieDTO?>(movie);
}
}
```